# Concurrent Execution Tests

*Ricart–Agrawala Distributed Mutual Exclusion*
*Implementation and observed results — Nodes A, B and C*

## 1. Test setup

The system was tested using three independent Go processes (A, B and C), each running as both a gRPC server and a client of the other two nodes. The nodes listen on localhost ports 50051, 50052 and 50053, respectively. Each process waits five seconds after becoming ready before requesting the critical section. A successful critical-section execution lasts two seconds, after which the node releases the section and sends any deferred replies.

The processes were launched from separate terminals with the Go race detector:

```
go run -race . -id=A
go run -race . -id=B
go run -race . -id=C
```

## 2. Observed concurrent test

In the recorded run, the requests were initiated close enough in time to overlap in the running system, but they were not synchronized to the exact same instant. The observed Lamport request timestamps were B = 1, C = 4 and A = 7. Therefore, the priority order in this run was B, then C, then A.

| Node | Observed critical-section interval | Relevant behaviour |
|---|---|---|
| B | 22:09:51–22:09:53 | Entered first; deferred replies to C and A; sent them after leaving. |
| C | 22:09:53–22:09:55 | Waited for B's deferred reply; entered second; deferred A's reply. |
| A | 22:09:55–22:09:57 | Waited for the required replies; entered last after C released the section. |

## 3. Interpretation against the requirements

### R1 — Specification

The run involved three peer nodes communicating through gRPC. Each node requested permission from the other nodes before entering the critical section. The critical section was represented by a log message and a two-second execution interval.

### R2 — Safety

The observed intervals did not overlap: B exited before C entered, and C exited before A entered. This run therefore provides evidence consistent with mutual exclusion: no two nodes were observed inside the critical section at the same time. A test run is evidence, not a formal proof for every possible execution.

### R3 — Liveness

All three requesting nodes eventually entered and exited the critical section. Deferred replies were sent when the node holding the critical section released it, allowing the waiting nodes to proceed. This is evidence of progress under the tested conditions, assuming reliable message delivery and no node failures.

## 4. Deferred-reply behaviour

The logs show the central Ricart–Agrawala mechanism. While B was in the critical section, it deferred the replies to C and A. After B exited, it sent both deferred replies. C could then complete its set of permissions and enter. While C was in the critical section, it deferred A's reply. After C exited, it sent that reply, allowing A to enter. This sequence illustrates how requests with lower priority wait without violating mutual exclusion.

## 5. Race-detector observation

The supplied logs did not contain any 'WARNING: DATA RACE' messages during this run with `go run -race`. This is a positive result for the tested execution, but it does not prove that the program is free of data races in all possible schedules.

## 6. Limitations and reproducibility notes

- Although all nodes use a five-second startup delay, each delay begins when its own process starts. Because the terminals were launched at different times, the requests were staggered rather than guaranteed to occur at precisely the same instant.
- The test demonstrates overlapping activity and deferred replies, but it is not a synchronized simultaneous-start experiment.
- The current main.go performs one automatic critical-section request per process and then keeps the gRPC server alive. It does not provide an interactive loop for repeatedly requesting the critical section.
- The implementation assumes reliable message delivery and that nodes do not crash while a request is pending. It does not implement crash recovery or fault tolerance.

## 7. Suggested conclusion for the report

The concurrent test with three nodes provided evidence that the Ricart–Agrawala implementation coordinates access to the critical section correctly under the tested conditions. All nodes eventually entered the section, deferred replies were released after the current holder exited, and the observed critical-section intervals did not overlap. No data-race warning appeared in the captured output. Since the requests were not synchronized to the exact same instant and only a finite number of runs were observed, these results should be presented as experimental evidence rather than a formal proof of safety and liveness.

## Implementation note

The reviewed main.go and node.go are consistent with the described test setup. The main program uses a five-second delay for each node, requests permission from all peers, simulates two seconds in the critical section, releases deferred replies, and leaves the server running. The node implementation uses Lamport timestamps, a timestamp-plus-node-ID priority comparison, mutex-protected shared state, and deferred gRPC replies.